

TBP Global Strategist

Technical Architecture Document

Version 1.0 — July 2026

Project Phase: Full System Architecture

Table of Contents

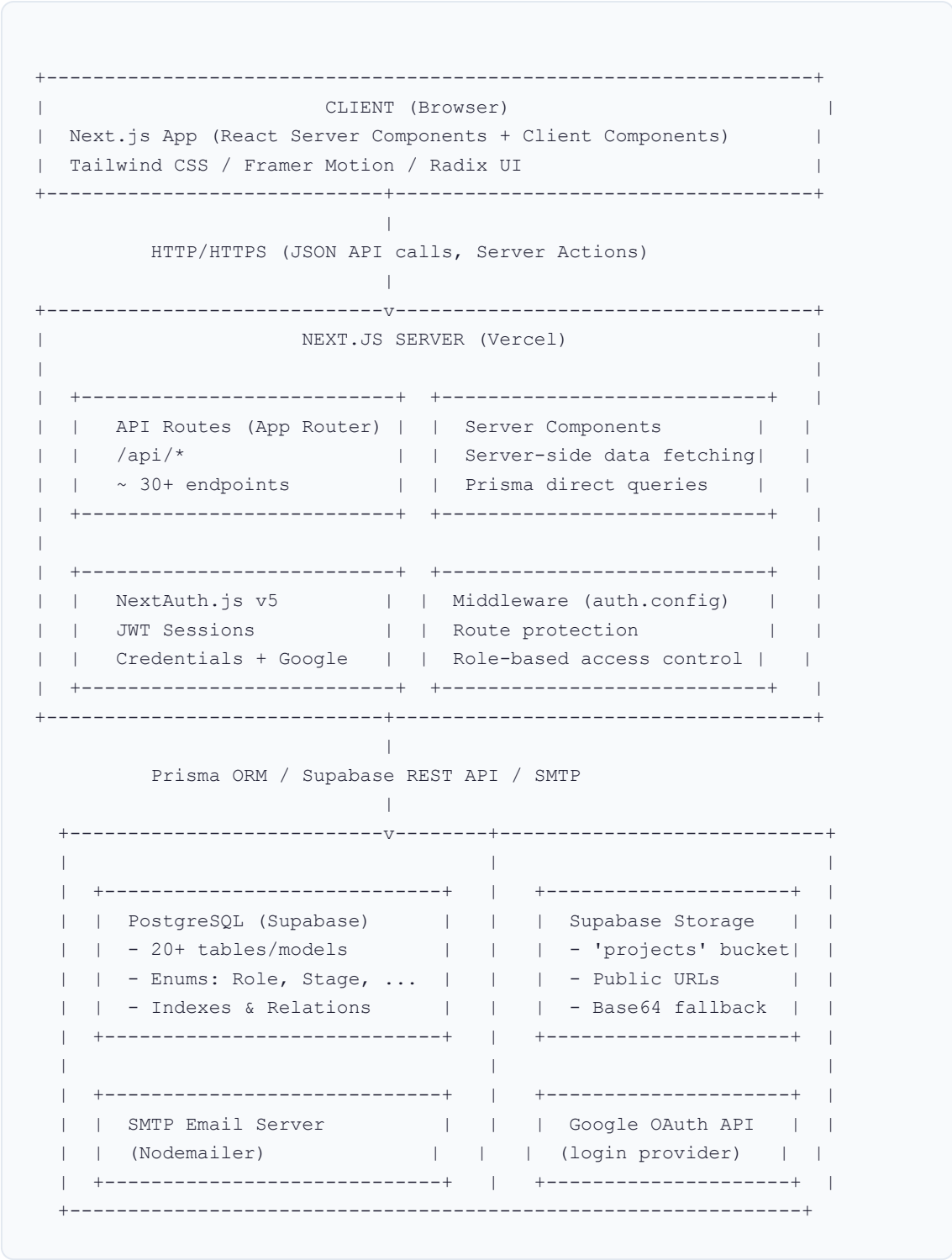
1. Technology Stack Overview
2. System Architecture Diagram
3. Hosting & Deployment (Vercel)
4. Database (PostgreSQL + Prisma)
5. File Storage (Supabase Storage)
6. Authentication (NextAuth.js)
7. API Layer (Next.js App Router)
8. Frontend Architecture
9. Email System (Nodemailer + SMTP)
10. Notifications System
11. Media & File Upload Pipeline
12. Environment Variables & Configuration
13. Data Flow & State Management
14. Security Architecture
15. Project Directory Structure

1. Technology Stack Overview

Layer	Technology	Version	Purpose
Framework	Next.js	16.2.6	Full-stack React framework (App Router)
Language	TypeScript	5.x	Type-safe development
Database	PostgreSQL (Supabase)	15+	Primary relational data store
ORM	Prisma	7.8.0	Database schema, migrations, type-safe queries
Authentication	NextAuth.js v5	beta	Auth with JWT sessions, Credentials + Google OAuth
File Storage	Supabase Storage	-	Project media, submission files, images
Email	Nodemailer	-	Transactional email via SMTP
Hosting	Vercel	-	Serverless deployment, edge network
Styling	Tailwind CSS	4.x	Utility-first CSS framework
Animation	Framer Motion	12.x	UI animations and transitions
UI Components	Radix UI (shadcn/ui)	-	Accessible headless UI primitives
Validation	Zod	-	Schema validation (API inputs, forms)
Icons	Lucide React	-	Icon library
Toasts	Sonner	-	Toast notifications
Date Handling	date-fns	-	Date formatting and manipulation
Password Hash	bcryptjs	-	Password hashing for credential auth

2. System Architecture Diagram

The following diagram illustrates the high-level architecture and data flow:



Architecture Phases

Phase	Component	Data Location	Technology
-------	-----------	---------------	------------

1. Presentation	UI Components	Client browser memory / React state	React Server/Client Components
2. API Gateway	Next.js API Routes	Vercel serverless functions	Next.js App Router handlers
3. Business Logic	Server-side code	Vercel serverless functions	TypeScript, Prisma queries
4. Data Access	Prisma ORM	Vercel → Supabase connection	Prisma Client + PostgreSQL adapter
5. Data Persistence	Database	Supabase (PostgreSQL)	Supabase managed PostgreSQL 15+
6. File Storage	Object Storage	Supabase Storage / Base64 fallback	Supabase S3-compatible API
7. External Services	Email, OAuth	SMTP server / Google	Nodemailer, NextAuth providers

3. Hosting & Deployment (Vercel)

Deployment Platform

- **Provider:** Vercel
- **Region:** Washington, D.C., USA (East) — iad1
- **Plan:** Production (Hobby/Pro)
- **Build Command:** `prisma generate && next build --webpack`
- **Output Directory:** Next.js default (`.next`)
- **Git Integration:** GitHub (`TBP-Initiative/tbpglobalstrategist`) — auto-deploys on push to `main`

Deployment Configuration

- No `vercel.json` — uses Vercel defaults for Next.js
- No custom `next.config.js` for production — minimal config in `next.config.ts`
- **Build Cache:** Enabled — restores from previous deployments
- **Node.js Version:** 24.x
- **Environment Variables:** Set via Vercel Dashboard (not in repo)

CI/CD Pipeline

1. Developer pushes to `main` on GitHub
2. Vercel detects the push via GitHub integration
3. Vercel runs `npm install` (539 packages)
4. Prisma generates client from schema
5. Next.js builds production bundle (webpack)
6. Output deployed to Vercel Edge Network
7. DNS alias updated to production domain
8. Build cache saved for next deployment

Edge Network

- Static pages (marked `o`) served from Vercel Edge Cache
- Dynamic pages (marked `f`) rendered serverlessly on demand
- API routes run as serverless functions per request

4. Database (PostgreSQL + Prisma)

Database Provider

- **Type:** PostgreSQL 15+
- **Hosted by:** Supabase
- **Connection:** Direct via `DATABASE_URL` environment variable
- **Adapter:** `@prisma/adapters-pg` (PrismaPg)
- **Connection Pool:** PgBouncer-compatible (port 6543)

ORM Layer

- **ORM:** Prisma Client 7.8.0
- **Generator Output:** `src/generated/prisma`
- **Config File:** `prisma.config.ts` (unified config)
- **Migrations Directory:** `prisma/migrations/`
- **Migration Tool:** Manual SQL (applied via Supabase SQL Editor)

Database Models & Data Locations

Model / Table	Purpose	Key Fields	Relations
User	User accounts & auth	id, name, email, role, passwordHash	→ StrategistProfile, Projects, Notifications, Submissions
StrategistProfile	Strategist professional info & progression	stage (enum), sector, city, country, bio, availability	→ User, → ExpertiseTag
Project	TBP project records	title, slug, status, budget, category, isFeatured	→ Organization, → User (creator), → Milestones, → Media, → Tags
Milestone	Project milestones/tasks	title, dueDate, completed, sortOrder, weight	→ Project
ProjectMedia	Project attachments	type (IMAGE/DOCUMENT/VIDEO), url, fileType, fileSize	→ Project
ProjectRelation	Project interconnections	relationType (RELATED/PARENT/CHILD/etc.)	→ Project (x2)
ProjectContributor	User-project assignments	role, joinedAt	→ Project, → User
Organization	Organizations/companies	name, slug, industry, size, location	→ OrganizationMember, → Project
OrganizationMember	Org membership	role	→ Organization, → User
WorkArea	Strategic work areas	name, slug (unique)	→ UserWorkArea

UserWorkArea	User-area assignments	-	→ User, → WorkArea
Submission	Report/task feedback uploads	stage, title, fileUrl, fileType, fileSize	→ User
Notification	In-app notifications	title, message, type, read, link	→ User
Message	User-to-user messages	content, read, senderId, receiverId	→ User (x2)
ActivityLog	Audit trail of actions	action (enum), entity, entityId, metadata	→ User
Tag	Taxonomy tags	name, slug	→ Project, → ExpertiseTag
ExpertiseTag	Strategist expertise labels	-	→ StrategistProfile, → Tag
Publication	User publications/articles	title, abstract, url, publisher	→ User
Achievement	User achievements	title, issuer, certificateUrl	→ User
Account	OAuth provider accounts	provider, providerAccountId, tokens	→ User (NextAuth)
Session	Database sessions (unused)	sessionToken, expires	→ User (NextAuth)
VerificationToken	Email verification tokens	identifier, token, expires	Standalone (NextAuth)

Enums

Enum	Values	Used By
Role	ADMIN, STRATEGIST, CORPORATE, ORGANIZATION_ADMIN, RESEARCHER, MODERATOR, PARTNER	User, OrganizationMember
ProgressionStage	CANDIDATE, STRATEGIST, CONTRIBUTOR, PROJECT_ALIGNED, SECTOR_LEAD, PAID_ADVISER	StrategistProfile
ProjectStatus	DRAFT, ACTIVE, COMPLETED, CANCELLED	Project
NotificationType	MESSAGE, PROJECT_INVITE, ORGANIZATION_INVITE, ACHIEVEMENT_UNLOCKED, PUBLICATION_APPROVED, SYSTEM	Notification
ActivityAction	CREATE, UPDATE, DELETE, LOGIN, LOGOUT, JOIN_ORGANIZATION, LEAVE_ORGANIZATION, JOIN_PROJECT, LEAVE_PROJECT, PUBLISH	ActivityLog

5. File Storage (Supabase Storage)

Storage Provider

- **Service:** Supabase Storage (S3-compatible)
- **Bucket:** `projects` (public bucket)
- **Access Pattern:** Public URLs via `/storage/v1/object/public/projects/{filename}`
- **Authentication:** Service role key (`SUPABASE_SERVICE_ROLE_KEY`) for uploads

Upload Flow

1. Client sends file as `multipart/form-data` to `POST /api/upload`
2. Server authenticates via NextAuth session
3. Server checks if `projects` bucket exists (auto-creates if 404)
4. Filename format: `projects/{timestamp}-{random}.{ext}`
5. File uploaded to Supabase Storage via REST API
6. On success, returns `{ url: "https://{supabase}/storage/v1/object/public/projects/{filename}" }`
7. On failure, falls back to base64 data URL

Data Stored in Supabase Storage

Data Category	File Types	Storage Path	Source
Project Images	jpg, png, webp, gif	<code>projects/{ts}-{rand}.{ext}</code>	Project create/edit forms
Project Media (docs)	pdf, docx, pptx, xlsx	<code>projects/{ts}-{rand}.{ext}</code>	Project media attachments
Submission Reports	pdf, docx, pptx, xlsx, csv, txt, rtf, odt, ods	<code>projects/{ts}-{rand}.{ext}</code>	Submission upload form
Profile Images	webp (optimized), jpg, png	<code>projects/{ts}-{rand}.{ext}</code>	Profile image upload (optimized client-side to max 400px WebP)

Image Optimization Pipeline

1. User selects image in browser
2. `src/lib/image-optimize.ts` reads with `FileReader`
3. Renders to `<canvas>` at max 400px width
4. Exports as WebP with 0.8 quality
5. Returns `{ dataUrl: "data:image/webp;base64, ... ", width, height, size }`
6. Optimized data URL sent to `POST /api/upload` for storage

6. Authentication (NextAuth.js)

Auth Provider

- **Library:** NextAuth.js v5 (beta)
- **Adapter:** `@auth/prisma-adapter` — stores accounts, sessions, verification tokens in DB
- **Session Strategy:** JWT (JSON Web Tokens)
- **Session Data:** `{ id, role, name, email, image }`

Authentication Providers

Provider	Type	Configuration
Credentials	Email + Password	Validated via Zod <code>loginSchema</code> , password hashed with <code>bcryptjs</code>
Google OAuth	OAuth 2.0	<code>GOOGLE_CLIENT_ID</code> + <code>GOOGLE_CLIENT_SECRET</code> env vars

Auth Flow

1. **Sign In:** User submits credentials → NextAuth validates → JWT created → Session established
2. **JWT Callback:** `token.id` and `token.role` set from user on sign-in
3. **Session Callback:** Token values mapped to `session.user.id` and `session.user.role`
4. **Authorization:** Middleware checks `auth.config.ts` — protects `/dashboard/*` and `/admin/*`

Route Protection

Route Pattern	Access
<code>/dashboard/*</code>	Authenticated users only
<code>/admin/*</code>	Authenticated + <code>role === "ADMIN"</code>
<code>/login</code> , <code>/register</code>	Redirect to dashboard if already logged in
Public routes (<code>/</code> , <code>/strategists</code> , <code>/projects</code> , etc.)	No authentication required

Role-Based Access Control (RBAC)

- **Roles:** ADMIN, STRATEGIST, CORPORATE, ORGANIZATION_ADMIN, RESEARCHER, MODERATOR, PARTNER
- **Admin routes:** `/dashboard/admin`, `/dashboard/users`, `/api/admin/*` — require ADMIN role
- **Stage-based access:** Project browsing and features may be gated by `ProgressionStage`
- **API-level checks:** Each API route verifies `session?.user` and checks `role` where needed

7. API Layer (Next.js App Router)

Architecture

- **Pattern:** Route Handlers (`route.ts` files) in App Router
- **Methods:** GET, POST, PATCH, DELETE as exported async functions
- **Response Format:** JSON via `NextResponse.json()`
- **Auth:** Each route calls `auth()` from `@/lib/auth` to verify session
- **Validation:** Zod schemas for request body validation

API Endpoints — Complete Inventory

Route	Methods	Auth	Purpose
<code>/api/profile</code>	GET, PATCH	User	Fetch/update own profile (name, image, strategistProfile fields)
<code>/api/register</code>	POST	None	Create new user account (name, email, password)
<code>/api/seed</code>	POST	Admin	Seed database with sample data
<code>/api/auth/[...nextauth]</code>	GET, POST	None	NextAuth handler (sign-in, sign-out, session, callbacks)
<code>/api/submissions</code>	GET, POST	User	List own submissions / create new submission
<code>/api/notifications</code>	GET, PATCH	User	List own notifications / mark as read/dismiss
<code>/api/upload</code>	POST	User	Upload file to Supabase Storage
<code>/api/uploads</code>	GET	User	List all files in Supabase Storage 'projects' bucket
<code>/api/projects</code>	GET, POST	User (POST)	List all projects / create new project
<code>/api/projects/search</code>	GET	None	Search projects by query

<code>/api/projects/featured</code>	GET	None	List featured projects
<code>/api/projects/categories</code>	GET	None	List project categories
<code>/api/projects/[id]</code>	GET, PATCH, DELETE	User (PATCH/DELETE)	Get/update/delete a specific project
<code>/api/projects/[id]/feature</code>	POST, DELETE	Admin	Toggle project featured status
<code>/api/projects/[id]/media</code>	POST, DELETE	Admin	Add/remove project media attachments
<code>/api/projects/[id]/milestones</code>	GET, POST	User	List/add project milestones
<code>/api/projects/[id]/milestones/[mid]</code>	PATCH, DELETE	User	Update/delete a milestone
<code>/api/projects/[id]/relations</code>	GET, POST, DELETE	User	Manage project-to-project relations
<code>/api/strategists</code>	GET	None	List all strategists with profiles
<code>/api/dashboard/stats</code>	GET	User	Aggregated dashboard statistics
<code>/api/admin/users</code>	GET	Admin	List all users with profiles and counts
<code>/api/admin/users/[id]</code>	GET	Admin	Get single user details
<code>/api/admin/users/[id]/role</code>	PATCH	Admin	Change user role
<code>/api/admin/users/[id]/stage</code>	PATCH	Admin	Change user progression stage (sends notification + email)
<code>/api/admin/users/[id]/areas</code>	GET, PATCH	Admin	Get/set user work area assignments
<code>/api/admin/users/[id]/password</code>	PATCH	Admin	Reset user password
<code>/api/admin/work-areas</code>	GET, POST	Admin	List all work areas / create new (auto-seeds defaults)

8. Frontend Architecture

Component Structure

- **Pages:** App Router page files (`page.tsx`) in `src/app/`
- **Server Components:** Default — data fetching at page level, passed as props to client components
- **Client Components:** Marked with `"use client"` — interactivity, state, effects, event handlers
- **Shared Components:** `src/components/shared/` — GlassCard, AnimatedSection, LoadingSpinner, etc.
- **UI Primitives:** `src/components/ui/` — shadcn/ui components (Button, Badge, Avatar, Dialog, etc.)
- **Dashboard Components:** `src/components/dashboards/` — StatsCard, ActivityFeed, NotificationsPanel, ProgressionPathway, SubmissionForm
- **Admin Components:** `src/components/admin/` — AssignAreasDialog, ChangeStageDialog, StageBadge

Routing Structure

Route	Type	Component
<code>/</code>	Static (O)	Home/Landing page
<code>/dashboard</code>	Dynamic (f)	Dashboard router (redirects based on role)
<code>/dashboard/individual</code>	Static (O)	Strategist individual dashboard
<code>/dashboard/admin</code>	Dynamic (f)	Admin panel
<code>/dashboard/profile</code>	Dynamic (f)	User profile page
<code>/dashboard/projects</code>	Dynamic (f)	User project management
<code>/dashboard/individual/browse</code>	Dynamic (f)	Browse projects (stage-gated)
<code>/strategists/[id]</code>	Dynamic (f)	Public strategist profile
<code>/projects/[id]</code>	Dynamic (f)	Public project detail page
<code>/login</code>	Static (O)	Login page

<code>/register</code>	Static (○)	Registration page
<code>/strategists</code>	Static (○)	Strategist directory
<code>/about, /contact, /insights, /initiatives, /summit, /research, /partners, /innovation</code>	Static (○)	Marketing/informational pages

State Management

- **Server State:** Fetched via `fetch()` in `useEffect` hooks (client components) or directly in server components via Prisma
- **Auth State:** `useSession()` from NextAuth — available globally via SessionProvider
- **Local UI State:** `useState`, `useReducer` in client components
- **No global state store** (no Redux, Zustand, etc.) — state is component-local or fetched from server
- **Form State:** Local `useState` objects in form components

Styling Architecture

- **Framework:** Tailwind CSS v4
- **Component Library:** shadcn/ui (Radix UI primitives + Tailwind styling)
- **Utility:** `cn()` from `src/lib/utils.ts` — merges Tailwind classes via `clsx` + `tailwind-merge`
- **Animation:** Framer Motion for page transitions, card animations, staggered list reveals
- **Icons:** Lucide React (tree-shakeable SVG icons)

9. Email System (Nodemailer + SMTP)

Configuration

Variable	Purpose
<code>SMTP_HOST</code>	SMTP server hostname
<code>SMTP_PORT</code>	SMTP port (default: 587)
<code>SMTP_SECURE</code>	Use TLS (true/false)
<code>SMTP_USER</code>	SMTP authentication username
<code>SMTP_PASS</code>	SMTP authentication password
<code>SMTP_FROM</code>	Sender email address (default: noreply@tbpglobal.com)

Email Usage

- **Stage Change Notifications:** When admin changes a user's stage, an email is sent with stage-specific message from `getStageMessage()`
- **Graceful Degradation:** If `SMTP_HOST` is not set, email sending is silently skipped with a console warning
- **Transport:** Nodemailer with `createTransport()` — instantiated once at module level

10. Notifications System

Architecture

- **Data:** Stored in `Notification` table in PostgreSQL
- **Creation:** `createNotification()` and `notifyAdmins()` in `src/lib/notifications.ts`
- **Retrieval:** `GET /api/notifications` — returns user's notifications ordered by newest first
- **Updates:** `PATCH /api/notifications` — mark as read or dismiss
- **UI:** Bell icon in nav + `NotificationsPanel` component on dashboard
- **Real-time:** Polls on page load (no WebSocket/polling currently)

Notification Triggers

Event	Recipient	Trigger Location
Profile updated	Admins	<code>PATCH /api/profile</code>
Stage changed	User + email	<code>PATCH /api/admin/users/[id]/stage</code>

Work areas assigned	User	PATCH /api/admin/users/[id]/areas
---------------------	------	-----------------------------------

11. Media & File Upload Pipeline

```

User selects file
  |
  v
[Client-side validation] — File type allowed? Size OK?
  |
  v
[Optional: Image Optimization]
  src/lib/image-optimize.ts
  - Reads via FileReader
  - Renders to canvas (max 400px)
  - Exports as WebP (0.8 quality)
  - Returns data:image/webp;base64,...
    |
    v
POST /api/upload (multipart/form-data)
  |
  v
[Server: Auth check] — session?.user required
  |
  v
[Supabase Storage]
  - Ensure 'projects' bucket exists
  - Upload as projects/{ts}-{rand}.{ext}
  - Return { url: "https://..." }
    |
    v
[Fallback: Base64 Data URL]
  - If Supabase unavailable
  - Return data URL (stored in DB)
    |
    v
[Create DB Record] (varies by context)
  - ProjectMedia record for project images
  - Submission record for report uploads
  - Profile image stored in User.image field

```


12. Environment Variables & Configuration

Variable	Required	Source	Used By
<code>DATABASE_URL</code>	Yes	Supabase / Local	Prisma (PostgreSQL connection)
<code>AUTH_SECRET</code>	Yes	Generated	NextAuth JWT encryption
<code>AUTH_TRUST_HOST</code>	Yes	Set to true	NextAuth host verification
<code>GOOGLE_CLIENT_ID</code>	Optional	Google Cloud Console	Google OAuth provider
<code>GOOGLE_CLIENT_SECRET</code>	Optional	Google Cloud Console	Google OAuth provider
<code>SUPABASE_URL</code>	Optional*	Supabase Dashboard	File uploads to Supabase Storage
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Optional*	Supabase Dashboard	File uploads (service-level access)
<code>SMTP_HOST</code>	Optional	Email provider	Nodemailer transport
<code>SMTP_PORT</code>	Optional	Email provider	Nodemailer port
<code>SMTP_SECURE</code>	Optional	Email provider	Nodemailer TLS
<code>SMTP_USER</code>	Optional	Email provider	Nodemailer auth
<code>SMTP_PASS</code>	Optional	Email provider	Nodemailer auth
<code>SMTP_FROM</code>	Optional	Email provider	Sender address

* Supabase variables are optional because the upload API falls back to base64 data URLs if storage is not configured. However, for production use, both are strongly recommended.

13. Data Flow & State Management

Data Flow Patterns

Pattern	Description	Example
Server Fetch → Client Render	Server component fetches data via Prisma, passes as props to client component	Profile page: <code>page.tsx</code> queries DB → passes <code>user</code> prop to <code>ProfileClient</code>

Client Fetch → State → Render	Client component fetches from API in useEffect, stores in useState, renders	Dashboard: fetches <code>/api/profile</code> and <code>/api/dashboard/stats</code>
Form → API → Refresh	User submits form → calls API → on success, refreshes data or navigates	EditProfileDialog → PATCH <code>/api/profile</code> → <code>router.refresh()</code>
Admin Action → Notification → UI Update	Admin performs action → creates notification → user sees in NotificationsPanel	Stage change → <code>notifyUser()</code> → notification appears on dashboard

State Locations

Data Type	Where It Lives	How It's Loaded
User session	NextAuth JWT + React Context	<code>useSession()</code> hook
Profile data	PostgreSQL (strategist_profiles table)	Server component Prisma query or <code>fetch(/api/profile)</code>
Dashboard stats	PostgreSQL (aggregated queries)	<code>fetch(/api/dashboard/stats)</code>
Projects list	PostgreSQL (projects table)	Server component Prisma query or <code>fetch(/api/projects)</code>
Notifications	PostgreSQL (notifications table)	<code>fetch(/api/notifications)</code>
Files/images	Supabase Storage (or base64 in DB)	Public URL from Supabase or data URL
Submissions	PostgreSQL (submissions table) + Supabase Storage	<code>fetch(/api/submissions)</code>
Work areas	PostgreSQL (work_areas + user_work_areas)	<code>fetch(/api/admin/work-areas)</code> or profile fetch
Activity feed	PostgreSQL (activity_logs table)	Server component Prisma query
Form state	Client component memory (useState)	User input
UI state (modals, tabs)	Client component memory (useState)	User interaction

14. Security Architecture

Authentication & Session Security

- **Password Hashing:** bcryptjs with salt rounds (default)
- **Session:** JWT-based (not database sessions), encrypted with `AUTH_SECRET`
- **Session Token:** Contains user `id` and `role` — never exposes `passwordHash`
- **OAuth:** Google OAuth 2.0 with client secret

API Security

- **Authorization:** Every protected API route calls `auth()` and checks session
- **Admin Routes:** Explicit role check — `(session.user as any).role !== "ADMIN"`
- **Input Validation:** Zod schemas for all PATCH/POST request bodies
- **File Upload:** No file type restriction on server (client-side only) — relies on Supabase storage security

Data Security

- **Database:** PostgreSQL hosted on Supabase with SSL/TLS
- **Storage:** Supabase Storage with service role key (server-side only)
- **Environment Variables:** Stored in Vercel Dashboard, not in repository
- **Prisma:** Generated client is gitignored (`/src/generated/prisma`)

Middleware Protection

- `auth.config.ts` exports `authConfig` used by NextAuth middleware
- Protects `/dashboard/*` — unauthenticated users are redirected to `/login`
- Protects `/admin/*` — non-admin users are redirected to `/dashboard`
- Redirects authenticated users away from `/login` and `/register`

15. Project Directory Structure

```
tbpglobalstrategist/
├─ prisma/
│  ├─ schema.prisma          # Database schema (20+ models, 5 enums)
│  ├─ config.ts              # Prisma unified config
│  ├─ seed.ts / seed.mjs     # Database seeding scripts
│  └─ migrations/            # SQL migration files
│     ├─ 20250708 ... work_areas/ # WorkArea + UserWorkArea tables
│     └─ 20250708 ... submissions/ # Submission table
├─ src/
│  └─ app/
```

```

├── (marketing)/           # Public-facing pages
│   ├── page.tsx          # Landing page
│   ├── strategists/      # Strategist directory + profiles
│   └── projects/         # Public project pages
├── api/                  # API route handlers (30+ endpoints)
│   ├── auth/             # NextAuth handler
│   ├── profile/          # User profile CRUD
│   ├── projects/         # Project CRUD + media + milestones
│   ├── submissions/      # Submission uploads
│   ├── upload/           # File upload to Supabase
│   ├── notifications/    # Notification system
│   ├── dashboard/        # Dashboard stats
│   ├── admin/            # Admin-only endpoints
│   └── ...               # Register, seed, strategists, etc.
├── dashboard/           # Dashboard pages
│   ├── individual/       # Strategist dashboard
│   ├── admin/            # Admin panel
│   ├── profile/          # Profile management
│   └── ...               # Projects, settings, users, etc.
├── components/
│   ├── ui/               # shadcn/ui primitives (Button, Badge, Dialog, etc.)
│   ├── shared/           # Shared components (GlassCard, AnimatedSection)
│   ├── dashboards/       # Dashboard-specific components
│   │   ├── stats-card.tsx
│   │   ├── activity-feed.tsx
│   │   ├── notifications-panel.tsx
│   │   ├── progression-pathway.tsx
│   │   └── submission-form.tsx
│   └── admin/            # Admin components
│       ├── assign-areas-dialog.tsx
│       ├── change-stage-dialog.tsx
│       └── stage-badge.tsx
├── lib/                  # Utility libraries
│   ├── auth.ts           # NextAuth configuration
│   ├── auth.config.ts    # Middleware auth config
│   ├── prisma.ts         # Prisma client singleton
│   ├── email.ts          # Nodemailer transport
│   ├── notifications.ts  # Create/manage notifications
│   ├── stage-messages.ts # Stage-specific messaging
│   ├── progression-pathway.ts # Stage data (tasks, outputs)
│   ├── image-optimize.ts # Client-side image optimization
│   ├── utils.ts          # cn(), formatDate(), truncate(), slugify()
│   └── validations.ts    # Zod validation schemas
├── types/                # TypeScript type declarations
│   ├── index.ts
│   └── next-auth.d.ts    # NextAuth session type augmentation
├── public/
│   └── docs/             # Generated documentation PDFs
├── next.config.ts        # Next.js configuration
├── prisma.config.ts      # Prisma configuration
├── package.json
├── tsconfig.json
├── tailwind.config.ts
└── .env                 # Local environment variables (gitignored)

```

Appendix: Data Lifecycle by Phase

Phase	Component	Data Stored	Location	Format	Retention
User Management	Auth, Profile	User accounts, roles, profile info, passwords (hashed)	PostgreSQL: User, StrategistProfile, Account, Session tables	Relational rows	Until account deletion
Authentication	NextAuth	JWT tokens, OAuth tokens, session data	JWT in cookie + PostgreSQL: Account, Session, VerificationToken	JWT + DB rows	Session expiry + token cleanup
Progression Pathway	Stage Management	Current stage, stage history (via ActivityLog)	PostgreSQL: StrategistProfile.stage + ActivityLog	Enum + JSON metadata	As long as user exists
Work Areas	Assignment	Work area definitions, user assignments	PostgreSQL: WorkArea, UserWorkArea	Relational rows	Until removed by admin
Project Management	Projects, Milestones	Project data, milestones, contributors, relations	PostgreSQL: Project, Milestone, ProjectContributor, ProjectRelation	Relational rows	Until project deletion
Project Media	Media Attachments	Images, videos, documents attached to projects	Supabase Storage (files) + PostgreSQL: ProjectMedia (metadata)	Binary files + metadata rows	Until media deletion
Report Submissions	File Upload	Uploaded reports, reflections, task feedback	Supabase Storage (files) + PostgreSQL: Submission (metadata)	PDF/DOCX/etc. + metadata rows	Until submission deletion
Notifications	In-App Alerts	Notification messages, read status	PostgreSQL: Notification	Relational rows	Until dismissed or auto-cleaned
Messaging	User Communication	Message content, read receipts	PostgreSQL: Message	Relational rows	Until message deletion
Activity Logging	Audit Trail	User actions, entity changes	PostgreSQL: ActivityLog	Relational rows	Indefinite (audit trail)

Email	Transactional	Stage change notifications, system emails	Sent via SMTP (not stored)	Text/HTML email	Not stored
UI State	Client Browser	Form inputs, modal states, local cache	Browser memory (React state)	JavaScript objects	Session lifetime (page refresh clears)
Static Assets	Documentation	PDF guides, documentation	Vercel (public/docs/)	Static files	Until removed from repo